

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №1 «Добыча корпуса документов»

Необходимо подготовить корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать его к себе на компьютер. В отчёте нужно указать источник данных.
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Разбить на документы.
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер «сырых» данных.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

1 Описание

Нужно проанализировать собранный корпус документов для его дальнейшего использования в лабораторных работах по курсу «Информационный поиск». Требуется дать описание выбранной темы и привести использованные источники, извлечь текстовое содержимое из сырых `html`-файлов, оценить существующие поисковые решения по данным источникам и собрать статистику.

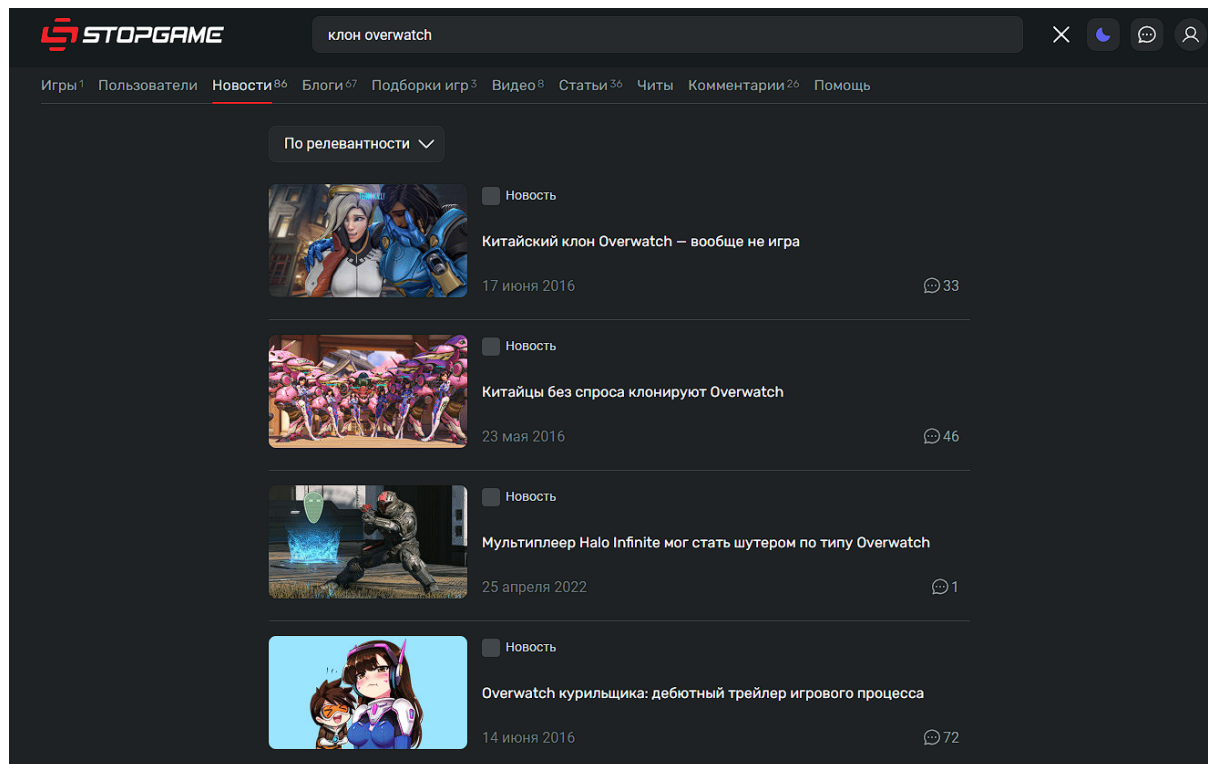
2 Исходный код

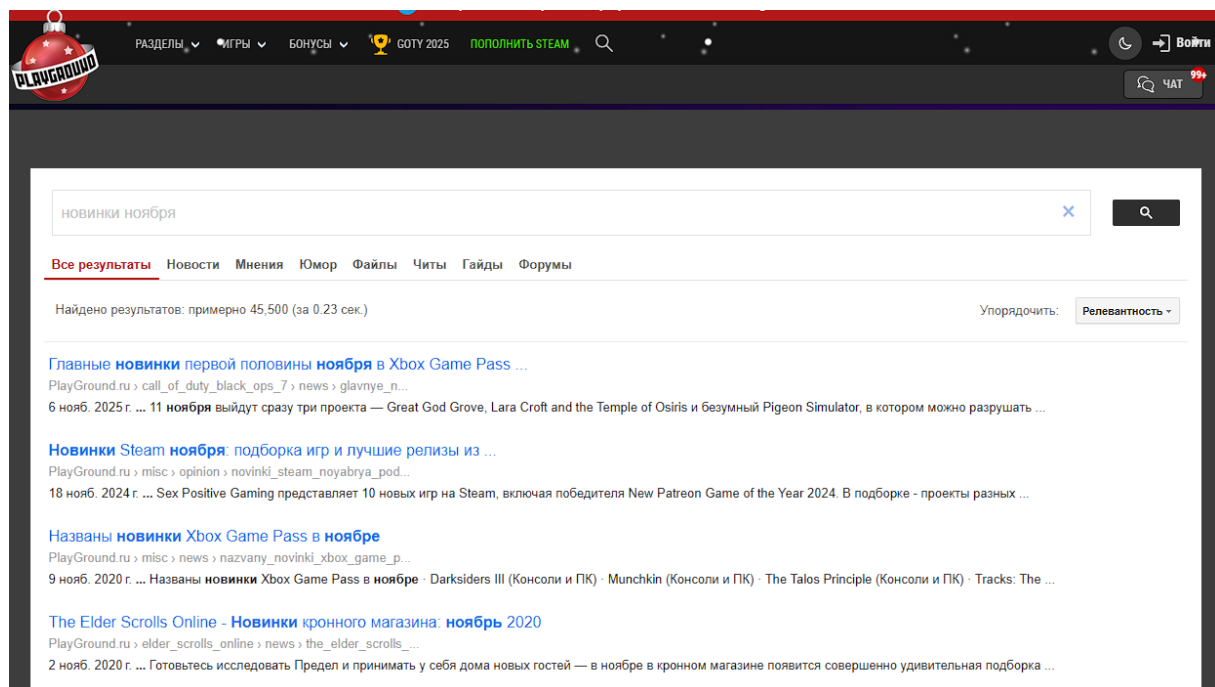
Источники данных. В качестве предметной области была выбрана игровая журналистика и видеоигры в целом. Корпус был сформирован из двух источников. С данных сайтов были собраны новостные статьи, а также видеоигровые рецензии:

- **StopGame.ru** (<https://stopgame.ru>) – крупный русскоязычный портал о видеоиграх, предоставляющий новости, обзоры, рецензии и видео.
- **PlayGround.ru** (<https://www.playground.ru>) – российский игровой портал, известный агрегацией новостей, статей, а также системой пользовательских рецензий и блогов.

Анализ существующих поисковых систем. Поиск по данным источникам может осуществляться следующим образом:

1. **Встроенный поиск на StopGame.ru и PlayGround.ru.** Есть возможность искать по контенту, опубликованному на этих сайтах.
2. **Поиск Google с оператором ‘site:’.** То есть, запрос ‘site:stopgame.ru лучшие командные шутеры’ выдаст конкретные статьи с сайта.





Характеристики и разметка текста. html-структура документов включает:

- В разделе `<head>` присутствуют теги `<title>`, `<meta name="description">`, `<meta name="keywords">`, а также разметка Open Graph (`og:`) и Schema.org для улучшения отображения в социальных сетях и поисковых системах.
- Текст статьи или обзора размещен внутри тегов `<body>`. Полезный контент окружен тегами (`<article>`, `<h1>`-`<h6>`, `<p>`), но присутствует большое количество разметки для навигации, рекламы, комментариев и сторонних скриптов.

Статистика корпуса. Статистика была собрана с помощью Python-скрипта

Статистика	Значение	Единица
Число документов	35274	Шт
Общий размер сырых данных	7116.87	Мб
Общий размер текста	368.44	Мб
Средний размер документа	0.20	Мб
Средний размер текста	7.57	Кб

Из сырых 7.12 Гб html-данных было извлечено лишь 368 Мб текста (примерно 5.05% от общего объема), это типично для современных веб-документов, где основную массу составляют разметка, стили и скрипты. Средний размер извлеченного из статьи

текста 7.57 Кб - это относительно короткие статьи, думаю, что это характерно для новостных и обзорных публикаций на игровых порталах.

3 Выводы

Выполнив первую лабораторную работу по курсу «Информационный поиск», на практике удалось понять, как сильно разнится сырой размер html-страницы и размер фактически значимых данных на ней. Работа с реальным корпусом, а также проведение его базового анализа (сбора статистики) закрепила это понятие, обсуждаемое на лекции.

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №2 «Поисковый робот»

Необходимо написать парсер на любом языке программирования.

- Единственным аргументом поисковому роботу подаётся путь до yaml-конфига, содержащий:
 - Данные для базы данных в секции db;
 - Данные для робота в секции logic: задержка между обкачкой страницы;
 - Любые другие данные, необходимые для реализации логики поискового робота.
- Сохранять в базе данных (например, MongoDB) документы со следующими полями:
 - url, нормализованный;
 - «сырой» html-текст документа;
 - название источника;
 - Дата обкачки документа в формате Unix time stamp.
- Поисковый робот можно остановить в любой момент и при повторном запуске робот должен начать с того документа, с которого он остановился;
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

4 Описание

Нужно разработать поискового робота, способного обходить веб-страницы и сохранять их содержимое в базу данных. В качестве базы данных была выбрана, упомянутая в условии лабораторной работы, **MongoDB** – документоориентированная NoSQL-система, которая подходит для хранения html-страниц.

Было принято решение использовать **sitemap** выбранных ресурсов для сбора URL. Sitemap – это XML-файл, который предоставляют сайты поисковикам для показа страниц, доступных для индексации. Sitemap содержит готовый список всех страниц сайта – не нужно рекурсивно обходить сайты. В sitemap указывается дата последнего изменения страницы (lastmod) - можно легко отслеживать изменения. Также URL уже нормализованы.

Необходимо удовлетворить условию и сделать так, чтобы робот мог останавливаться и продолжать работу, а также переобкачивать страницы. Также нужен конфигурационный yaml-файл.

5 Исходный код

Робот написан на Python с использованием: `requests` для HTTP-запросов, `PyYAML` для чтения конфигурации, `pymongo` для работы с MongoDB, `xml.etree.ElementTree` для парсинга XML-файлов sitemap.

yaml файл. При работе с sitemap источника stopgame.ru была замечена особенность: указанные в нем URL не соответствовали фактическим: указано `https://stopgame.ru/news/64976/`, а реальные были вида `https://stopgame.ru/newsdata/64976/`. Для этого в конфигурационный файл была добавлена логика `url_transforms`, позволяющая при помощи регулярных выражений приводить ссылки одного вида к другому, чтобы решить эту проблему. Поля конфигурационного файла `robot.yml`:

```
! robot.yml
1  # Конфигурация базы данных MongoDB
2  db:
3      connection_string: "mongodb://admin:password123@localhost:27017/"
4      documents_database: "documents_db"
5
6  # Настройки состояния робота
7  state:
8      state_dir: "crawler_state"
9
10 # Логика работы краулера
11 logic:
12     delay_between_requests: 5
13     max_urls_per_sitemap: 20000
14
15     url_transforms:
16     - sitemap_pattern: "https://stopgame.ru/sitemap/news_1.xml"
17       pattern: "^https://stopgame\\.ru/news/(.+)$"
18       replacement: "https://stopgame.ru/newsdata/\\1"
19
20     sitemaps:
21     - "https://stopgame.ru/sitemap/reviews_1.xml"
22     - "https://www.playground.ru/sitemap/news/list.xml"
23     - "https://stopgame.ru/sitemap/news_1.xml"
24     - "https://stopgame.ru/sitemap/news_2.xml"
```

- `db` – настройки БД:

- `connection_string` – строка подключения к MongoDB
- `documents_database` – название БД
- `state` – управление состоянием:
 - `state_dir` – директория для хранения файлов состояния
- `logic` – секция логики работы:
 - `delay_between_requests` – задержка между загрузками страниц в секундах
 - `max_urls_per_sitemap` – максимальное количество URL, обрабатываемых из одного sitemap (0 — если не нужны ограничения)
 - `interval_seconds` – интервал переобкатки
 - `sitemaps` – список URL sitemap-файлов для обработки
 - `url_transforms` – список правил трансформации URL

Сохранения состояния. Состояние робота хранится в вспомогательных json-файлах:

- `queue.json` – текущая очередь URL на обкатку/переобкатку. При остановке робота очередь сохраняется. Так что робот продолжит работу с того места, где остановился.
- `sitemap_cache.json` – кэш данных из sitemap. Это словарь, ключ — URL страницы, значение — дата последнего изменения (`lastmod`) из sitemap. Он нужен чтобы при возобновлении работы заново не парсить все URL из sitemap. Обновляется периодически.

Оба файла сохраняются в `state_dir`. При нажатии (Ctrl+C) робот сохраняет состояние и завершает работу.

Алгоритм работы робота. Робот находится в бесконечном цикле:

Если очередь пуста (первый запуск или все URL обработаны):

1. Для каждого sitemap из конфигурации:
 - Загружается и парсится XML-файл
 - Извлекаются URL и даты последнего изменения (`lastmod`)
 - Применяются правила трансформации (если заданы)

- Данные сохраняются в `sitemap_cache.json`

Для каждого URL из кэша sitemap:

1. Если URL отсутствует в БД – добавляется в очередь
2. Для уже обкаченных URL – проверяются два условия:
 - Прошло ли достаточно времени с последней обкачки (сравнение с `interval_seconds`)
 - Изменилась ли страница (сравнение `lastmod` из sitemap с временем последней обкачки из БД)
 - Если оба условия верны - нужно переобкачать

Обработка очереди:

1. URL извлекаются из очереди по одному
2. Для каждого URL:
 - Загружается html-файл
 - Извлекается название источника (домен из URL)
 - Формируется документ БД с полями:
 - `_id` – уникальный числовой идентификатор
 - `url` – нормализованный URL (после трансформации)
 - `html` – сырой html-код
 - `source_name` – название источника
 - `crawl_time` – время обкачки в формате Unix timestamp
 - Документ сохраняется в БД
3. Выжидаем паузу `delay_between_requests`

Когда очередь пуста робот возвращается к обновлению кэша sitemap, начиная новый цикл. Если изменений в sitemap нет – робот переходит в режим ожидания до появления новых данных или изменения существующих.

Для выбранных источников в файлах `robots.txt` не было обнаружено вежливого интервала для обкачки, так что она осуществлялась с интервалом в 5 секунд (приходилось оставлять робота работать по ночам)

6 Выводы

Выполнять лабораторную работу было интересно, до этого мне не приходилось заниматься обкачкой сайтов. Было интересно узнать, что для сайтов существует свой информационный файл для роботов `robots.txt`, а также карта всех страниц `sitemap`. Обеспечение сохранения состояния между запусками было занимательной задачей, пришлось придумать собственную схему, правда, наверняка, не самую эффективную. Соблюдение условных правил вежливой обкачки позволило обеспечить стабильную работу робота на протяжении многих непрерывных часов.

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №3 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №3 «Токенизация»

Нужно реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Для этого потребуется выработать правила, по которым текст делится на токены. Необходимо описать их в отчёте, указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы поправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные:

- Количество токенов.
- Среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

7 Описание

Нужно разработать программу для токенизации текстовых документов, которая, разобьет текст на токены, приведет их к нижнему регистру и удалит стоп-слова. Токенизация это первый этап в процессе индексации документов для информационного поиска. Нужно разработать правила разбиения текста на токены, реализовать алгоритм токенизации, провести анализ результатов работы и собрать статистику: количество токенов, среднюю длину токена, время выполнения программы и скорость обработки данных.

8 Исходный код

Для разработки программ использовался язык программирования С. В конечном итоге был получен набор утилит:

1. `./tokenizer <in_dir> <out_dir>` — разбивает текст на отдельные токены
2. `./normalizer <in_dir> <out_dir>` — приводит текст к нижнему регистру и заменяет букву «ё» на «е»
3. `./stopwords <in_dir> <out_dir>` — удаляет стоп-слова (неинформативные слова)
4. `./pipeline <in_dir> <out_dir>` — применяет весь пайплайн обработки одновременно, а также собирает статистику

Были написаны небольшие библиотеки для токенизации, нормализации и удаления стоп слов из текста. В процессе отладки было удобно применять эти этапы поочередно, чтобы верифицировать результат, поэтому были выделены отдельные утилиты для каждого этапа, а также общая, применяющая весь пайплайн обработки к текстам сразу и собирающая статистику.

Работа с UTF-8. UTF-8 — это многобайтовая кодировка, где один символ может занимать от 1 до 4 байтов. Если читать файл как обычную строку `char*`, мы не можем напрямую обращаться к символам по индексу (например, `text[i]`), потому что один символ может быть разбит на несколько байтов.

Поэтому был написан набор функций для преобразования входного набора байт в массив `wchar_t`. Тип `wchar_t` представляет собой широкий символ, который хранит каждый символ Юникода отдельно. Это позволило корректно обращаться к символам по индексу, а также использовать функции для определения длины текста, нахождения символов (`wcslen`, `wcschr`).

Токенизация. Библиотека `tokenizer_lib.c` реализует функцию токенизации текста на широких символах `wchar_t`. Токенизация выполняется путем разделения текста на токены с удалением разделителей и заменой их одиночными пробелами между токенами.

1. **Определяем разделители:** пробелы, знаки пунктуации и специальные символы
2. **Проходим по тексту** символ за символом
3. **Находим токены:**

- Все между разделителями = один токен
- Последовательность разделителей = один пробел между токенами
- Разделители на краях отбрасываются

4. **Сохраняем результат:** токены через один пробел, без лишних пробелов

```

1 | static
2 | bool is_delimiter( wchar_t c )
3 | {
4 |     if ( iswspace( c ) || iswpunct( c ) )
5 |         return 1;
6 |
7 |     switch ( c )
8 |     {
9 |         case L'«': case L'»': case L',': case L'“':
10 |         case L'”': case L'<': case L'>': case L'-' :
11 |         case L'_' : case L'...' :
12 |             return 1;
13 |         default:
14 |             return 0;
15 |     }
16 | }
```

Нормализация текста. Библиотека `normalizer_lib.c` реализует функцию нормализации текста на широких символах `wchar_t`. Нормализация выполняется путем преобразования текста в нижний регистр и замены буквы "ё" на "е"

1. Проходим по тексту символ за символом

2. Нормализуем символы:

- Русские буквы в верхнем регистре преобразуются в нижний
- Английские буквы в верхнем регистре преобразуются в нижний
- Буква "ё" преобразуется в "е"
- Цифры остаются без изменений

```

1 | static
2 | bool is_russian_letter( wchar_t c )
3 | {
```

```

4      if ( (c >= L'a' && c <= L'я') ||
5           (c >= L'A' && c <= L'Я') ||
6           (c == L'ё' || c == L'Ё') )
7          return true;
8
9      return false;
10 }
11
12 static
13 bool is_english_letter( wchar_t c )
14 {
15     if ( (c >= L'a' && c <= L'z') ||
16          (c >= L'A' && c <= L'Z') )
17         return true;
18
19     return false;
20 }
21
22 // normalization process in code
23 // =====
24 // ...
25 if ( is_russian_letter( cur_char ) )
26 {
27     if (cur_char >= L'A' && cur_char <= L'Я')
28         cur_char = L'a' + (cur_char - L'A');
29     else if ( (cur_char == L'Ё') ||
30              (cur_char == L'ё') )
31         cur_char = L'e';
32 } else if ( is_english_letter( cur_char ) )
33     cur_char = L'a' + (cur_char - L'A');
34 // ...
35 // =====

```

Удаление стоп-слов. Библиотека `stopwords_lib.c` реализует функцию удаления стоп-слов из текста на широких символах `wchar_t`. Удаление выполняется путем выявления токенов, входящих в фиксированный список стоп-слов. Важно отметить, что тут и далее в процессе подготовки текста было принято решение оставлять английские токены без изменений. В нашем случае (статьи видеоигровой тематики на русском языке) английские слова редки, и как правило являются названиями проектов.

1. Проходим по тексту токен за токеном

2. Фильтруем слова:

- Если слово присутствует в списке стоп-слов – оно удаляется
- Если слово отсутствует в списке – оно сохраняется в выходном буфере

```
1 static const wchar_t* RUSSIAN_STOPWORDS[] =
2 {
3     L"и", L"в", L"во", L"не", L"что", L"он", L"на", L"я", L"с", L"со",
4     L"как", L"а", L"то", L"все", L"она", L"так", L"его", L"но", L"да",
5     L"ты", L"к", L"у", L"же", L"вы", L"за", L"бы", L"по", L"только",
6     L"ее", L"мне", L"было", L"вот", L"от", L"меня", L"еще", L"нет",
7     L"о", L"из", L"ему", L"теперь", L"когда", L"даже", L"ну", L"ли",
8     L"если", L"уже", L"или", L"ни", L"быть", L"был", L"него", L"до",
9     L"вас", L"нибудь", L"опять", L"уж", L"вам", L"ведь", L"там", L"потом",
10    L"себя", L"ничего", L"ей", L"может", L"они", L"тут", L"где", L"есть",
11    L"надо", L"ней", L"для", L"мы", L"тебя", L"их", L"чем", L"была",
12    L"сам", L"чтоб", L"без", L"будто", L"чего", L"раз", L"тоже", L"себе",
13    L"под", L"будет", L"ж", L"тогда", L"кто", L"этот", L"того", L"потому",
14    L"этого", L"какой", L"совсем", L"ним", L"здесь", L"этом", L"один",
15    L"почти", L"мой", L"тем", L"чтобы", L"нее", L"сейчас", L"были",
16    L"куда", L"зачем", L"всех", L"никогда", L"можно", L"при", L"наконец",
17    L"два", L"об", L"другой", L"хоть", L"после", L"над", L"больше",
18    L"тот", L"через", L"эти", L"нас", L"про", L"всего", L"них", L"какая",
19    L"много", L"разве", L"три", L"эту", L"моя", L"впрочем", L"хорошо",
20    L"свою", L"этой", L"перед", L"иногда", L"лучше", L"чуть", L"том",
21    L"нельзя", L"такой", L"им", L"более", L"всегда", L"конечно", L"всю",
22    L"между"
23 };
24
25 static const int RUSSIAN_STOPWORDS_COUNT = sizeof(RUSSIAN_STOPWORDS) /
26     sizeof(RUSSIAN_STOPWORDS[0]);
27
28 static
29 bool is_stopword( const wchar_t *word )
30 {
31     if ( !word || wcslen(word) == 0 )
32         return false;
33
34     for ( int i = 0; i < RUSSIAN_STOPWORDS_COUNT; i++ )
```

```

34 | {
35 |     if ( wcsncmp( word, RUSSIAN_STOPWORDS[i] ) == 0 )
36 |         return true;
37 | }
38 |
39 | return false;
40 | }

```

Результаты. В результате токенизации было получено **22 364 175** токенов. Средняя длина одного токена составила **6.49** символа.

Производительность:

- **Скорость обработки данных: 2.24 Мб/сек.**
- **Скорость токенизации: 135 996 токенов/сек.**

Достигнутую производительность нельзя считать оптимальной. За счет внедрения параллельной обработки файлов можно было бы достигнуть кратного улучшения производительности.

Общее время работы полного пайплайна составило **164.45** секунды.

Результат токенизации.

Where Winds Meet: Обзор самой эпической игры года Где встречаются The Witcher 3, Genshin Impact, Assassinss Creed и Ghost of Tsushima Игру «Там, где встречаются ветра» стоило бы назвать «Там, где встречаются тренды из всех игр сразу». В этом китайском чуде от Everstone Studio и NetEase Games можно разглядеть не только те проекты, которые я вынес в подзаголовок обзора, но и другие, включая Black Myth: Wukong и Sekiro: Shadows Die Twice. Но вот что удивительно — при этом в Where Winds Meet полно и своих фишек. Ну где ещё вы можете вступить во фракцию воинов-алкоголиков, сыграть в шахматы с лидером бандитского лагеря, кидаться медведями, драться с гусями, кататься на них верхом, а потом лечить гусей (нет, не от ПТСР, но было бы логично).

where winds meet обзор самой эпической игры года встречаются the witcher 3
genshin impact assassins creed ghost of tsushima игру встречаются ветра стоило
назвать встречаются тренды игр сразу китайском чуде everstone studio netease
games разглядеть те проекты которые вынес подзаголовок обзора другие вклю-
чая black myth wukong sekiro shadows die twice удивительно where winds meet
полно своих фишек можете вступить фракцию воинов алкоголиков сыграть
шахматы лидером бандитского лагеря кидаться медведями драться гусями ка-
таться верхом лечить гусей нет птер логично

9 Выводы

Выполнение лабораторной работы позволило разобраться в том, как работать с текстами в кодировке UTF-8 в коде на C, а также познакомиться с пайплайном обработки данных для дальнейшего приготовления индекса. Было интересно наблюдать за тем, как входной текст преобразуется до отдельных токенов шаг за шагом: токенизация -> нормализация -> удаление стоп слов.

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №4 «Стемминг»

Добавить в созданную поисковую систему стемминг.

Стемминг можно добавлять на этапе индексации, можно на этапе выполнения поискового запроса.

10 Описание

Нужно разработать программу для стемминга полученных токенов. Реализуем ее в виде библиотеки и включим в утилиту полного пайплайна.

Стемминг – процесс приведения слова к его основе или корневой форме (стему) путем отбрасывания словоизменятельных и словообразовательных аффиксов. Стемминг позволяет группировать однокоренные слова, сокращая размер словаря и повышая полноту поиска за счет учета различных грамматических форм одного и того же слова.

Итоговый стеммер реализован для русского языка на основе классического алгоритма Портера. Алгоритм является эвристическим и состоит из последовательности правил, применяемых к слову в определенном порядке. Правила опираются на понятие регионов слова:

1. **RV** – регион, начинающийся после первой гласной в слове
2. **R1** – регион после первого сочетания согласная + гласная
3. **R2** – регион после первого сочетания согласная + гласная внутри R1

Регионы определяются для каждого слова, и удаление окончаний разрешено только в том случае, если окончание лежит внутри соответствующего региона. Это позволяет сохранять основу слова и не удалять значимые части.

11 Исходный код

Для стемминга и финальной очистки токенов разработаны две библиотеки, а также отдельные утилиты:

1. `./stemmer.c <in_dir> <out_dir>` – реализует алгоритм стемминга для русского языка по методу Портера.
2. `./finalizer.c <in_dir> <out_dir>` – удаляет однобуквенные слова (кроме цифр) из текста.

Стемминг. Библиотека `stemmer_lib.c` последовательно применяет к слову ряд правил, разделенных на 9 шагов. Каждое правило проверяет наличие определенного окончания и удаляет его, если оно находится в допустимом регионе.

1. **Определение регионов:** вычисляются RV, R1, R2.
2. **Шаг 1: Perfective gerund** – удаляет окончания прошедшего времени деепричастий (*ивши, ывши, ив, ыв, виш, в*).
3. **Шаг 2: Adjective** – удаляет окончания прилагательных и причастий (*ее, ие, ые, ое, ими, ыми, ей, ий, ...*).
4. **Шаг 3: Participle** – удаляет окончания причастий (*ивш, ывш, ующ, ем, нн, вш, ющ, щ*).
5. **Шаг 4: Reflexive** – удаляет возвратные окончания (*ся, съ*).
6. **Шаг 5: Verb** – удаляет личные глагольные окончания (*ила, ыла, ена, ейте, уйте, ...*).
7. **Шаг 6: Noun** – удаляет падежные окончания существительных (*а, ев, ов, ие, ье, е, иями, ями, ...*).
8. **Шаг 7: Superlative** – удаляет превосходную степень (*ейш, ейше*).
9. **Шаг 8: Undouble н** – убирает удвоенную «н» в конце основы.
10. **Шаг 9: Soft sign** – удаляет мягкий знак в конце слова.

```

1 static
2 void russian_stemmer( wchar_t *word )
3 {
4     if ( !is_russian_word( word ) )
5         return;
6
7     if ( wcslen( word ) < 3 )
8         return;
9
10    size_t r1, r2, rv;
11    find_regions( word, &r1, &r2, &rv );
12    size_t len = wcslen(word);
13
14    // Шаг1: Perfective gerund
15    if ( has_ending( word, L"ИВШИ" ) ||
16         has_ending( word, L"ЫВШИ" ) ||
17         has_ending( word, L"ИВ" ) ||
18         has_ending( word, L"ЫВ" ) )
19    {
20        if ( len - 2 >= rv )
21        {
22            remove_ending( word, L"ИВШИ" );
23            remove_ending( word, L"ЫВШИ" );
24            remove_ending( word, L"ИВ" );
25            remove_ending( word, L"ЫВ" );
26        }
27    } else if ( has_ending( word, L"ВШИ" ) ||
28                has_ending( word, L"В" ) )
29    {
30        if ( (len - 1 >= rv) &&
31             (has_ending( word, L"ВШИ" ) || has_ending( word, L"В" ) ) )
32        {
33            remove_ending( word, L"ВШИ" );
34            remove_ending( word, L"В" );
35        }
36    }
37
38    // Шаг2: Adjective
39    const wchar_t *adjective_endings[] = {
40        L"ее", L"ие", L"ые", L"ое", L"ими", L"ыми", L"ей", L"ий",
41        L"ый", L"ой", L"ем", L"им", L"ым", L"ом", L"его", L"ого",

```

```

42     L"ему", L"ому", L"их", L"ых", L"ую", L"юю", L"ая", L"яя",
43     L"ою", L"ею", NULL
44 };
45
46 for ( int i = 0; adjective_endings[i] != NULL; i++ )
47 {
48     if ( has_ending( word, adjective_endings[ i ] ) )
49     {
50         if ( len - wcslen( adjective_endings[ i ] ) >= r1 )
51             remove_ending( word, adjective_endings[ i ] );
52
53         break;
54     }
55 }
56
57 // Шаг3: Participle
58 if ( has_ending( word, L"ивш" ) ||
59     has_ending( word, L"ывш" ) ||
60     has_ending( word, L"ующ" ) )
61 {
62     if ( len - 3 >= rv )
63     {
64         remove_ending( word, L"ивш" );
65         remove_ending( word, L"ывш" );
66         remove_ending( word, L"ующ" );
67     }
68 } else if ( has_ending( word, L"ем" ) ||
69     has_ending( word, L"нн" ) ||
70     has_ending( word, L"вш" ) ||
71     has_ending( word, L"ющ" ) ||
72     has_ending( word, L"щ" ) )
73 {
74     if ( len - 2 >= rv )
75     {
76         remove_ending( word, L"ем" );
77         remove_ending( word, L"нн" );
78         remove_ending( word, L"вш" );
79         remove_ending( word, L"ющ" );
80         remove_ending( word, L"щ" );
81     }
82 }

```

```

83 || // =====
84 || // other steps...

```

Финальная подготовка токенов. После применения стемминга все равно в списке токенов оказались токены, которые сократились до 1 символа. Для решения этой проблемы была написана небольшая библиотека `finalizer_lib.c` реализующая функцию удаления однобуквенных слов из текста. Исключение составляют однобуквенные цифры, которые могут быть частью номеров или обозначений.

Результат стемминга.

Where Winds Meet: Обзор самой эпической игры года Где встречаются The Witcher 3, Genshin Impact, Assassins Creed и Ghost of Tsushima Игру «Там, где встречаются ветра» стоило бы назвать «Там, где встречаются тренды из всех игр сразу». В этом китайском чуде от Everstone Studio и NetEase Games можно разглядеть не только те проекты, которые я вынес в подзаголовок обзора, но и другие, включая Black Myth: Wukong и Sekiro: Shadows Die Twice. Но вот что удивительно — при этом в Where Winds Meet полно и своих фишек. Ну где ещё вы можете вступить во фракцию воинов-алкоголиков, сыграть в шахматы с лидером бандитского лагеря, кидаться медведями, драться с гусями, кататься на них верхом, а потом лечить гусей (нет, не от ПТСР, но было бы логично).

where winds meet обзор эпическ игр год встреч the witcher 3 genshin impact assassins creed ghost of tsushima игру встреч ветр ст назв встреч тренд игр сразу китайск чуд everstone studio netease games разгляд те проект котор вынес подзаголовок обзор друг включ black myth wukong sekiro shadows die twice удивительн where winds meet полн св фишек может вступ фракц воин алкоголик сыгр шахмат лидер бандитск лагерь кид медвед др гус кат верх леч гус нет птср логичн

12 Выводы

Написание стеммера для слов является упрощенным вариантом сокращения токенов, по сравнению с лемматизацией. Была предпринята попытка написать свой небольшой стеммер. В результате его работы некоторые токены все равно сводились к 1 символу, эту проблему было принято решить удалением таких токенов из рассмотрения. Учитывая выбранную предметную область и российские источники со статьями на русском языке принято решение применять стемминг только для слов на русском языке, а названия игр оставить без изменений.

На примере из конца этого отчета очень ярко видно, что текст сильно сократился и стал практически нечитаемым для человека, однако это позволит эффективно использовать полученные токены в процессе индексации за счет сокращения их размера.

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №5 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №5 «Закон Ципфа»

Для своего корпуса необходимо построить график распределения терминов по частотностям в логарифмической шкале, наложить на этот график закон Ципфа. Объяснить причины расхождения.

13 Описание

Нужно построить график распределения терминов в полученном корпусе по частотностям в логарифмической шкале и наложить на него график закона Ципфа.

Закон Ципфа. Закон Ципфа — эмпирический закон, описывающий распределение. Он утверждает, что если все слова некоторого корпуса упорядочить по убыванию частоты их использования, то частота f_r r -го по популярности слова будет приблизительно обратно пропорциональна его рангу r .

Математически закон записывается как:

$$f_r \approx \frac{C}{r^\alpha},$$

где:

- f_r — частота слова, имеющего ранг r ,
- r — ранг слова (1 для самого частого, 2 для следующего и т.д.),
- C — константа, часто близкая к частоте самого распространенного слова,
- α — параметр, близкий к 1 для естественных языков (классический закон Ципфа).

В логарифмической шкале зависимость становится линейной:

$$\log(f_r) \approx \log(C) - \alpha \cdot \log(r).$$

Таким образом, на графике в двойных логарифмических координатах ($\log r$ по оси X, $\log f_r$ по оси Y) эмпирические данные, подчиняющиеся закону Ципфа, должны лечь приблизительно на прямую линию с отрицательным наклоном $-\alpha$.

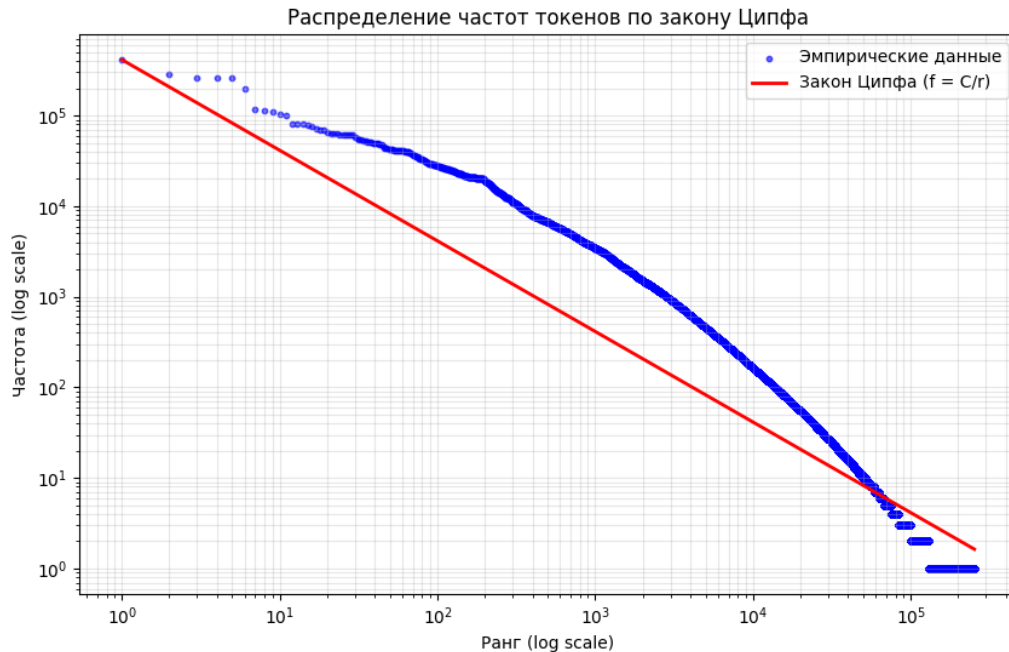
Назначение и применение. Анализ распределения слов по закону Ципфа используется для:

- Исследования статистических свойств языка и текстовых корпусов
- Верификации качества построения корпуса (естественные тексты обычно следуют этой закономерности).
- Выявления особенностей стиля или предметной области по отклонениям от классического распределения

- Оптимизации в информационном поиске и лингвистике (сжатие данных, построение словарей)

14 Исходный код

Для построения графика по закону Ципфа был написан скрипт на Python.



В результате видим, что частотность слов в нашем корпусе (22.2 млн токенов) подчиняется степенному закону, то есть вероятность слова примерно обратно пропорциональна его рангу. Отклонения от идеальной прямой в области высоких рангов (редкие слова) характерны для реальных данных и могут быть связаны с ограниченным объемом выборки.

```
1 # =====
2 # ...
3 for filename in sorted(os.listdir(dir_path)):
4     if filename.endswith(".txt"):
5         file_path = os.path.join(dir_path, filename)
6         try:
7             with open(file_path, 'r', encoding='utf-8') as f:
8                 content = f.read().strip()
9                 if content:
10                     tokens = content.split()
11                     all_tokens_counter.update(tokens)
12 except Exception as e:
13     print(f"Ошибка при чтении файла {filename}: {e}")
```

```

14 | # ...
15 | # =====
16 |
17 | sorted_frequencies = sorted(all_tokens_counter.values(), reverse=True)
18 |
19 | ranks = list(range(1, len(sorted_frequencies) + 1))
20 | frequencies = sorted_frequencies
21 |
22 | C = frequencies[0]
23 | zipf_frequencies = [C / r for r in ranks]
24 |
25 | plt.figure(figsize=(10, 6))
26 |
27 | plt.scatter(ranks, frequencies, s=10, alpha=0.6, label='данные', color
    |         ='blue')
28 |
29 | plt.plot(ranks, zipf_frequencies, 'r-', linewidth=2, label='Ципф (f =
    |         C/r)')
30 |
31 | plt.xscale('log')
32 | plt.yscale('log')
33 | plt.xlabel('Ранг (log scale)')
34 | plt.ylabel('Частота (log scale)')
35 | plt.grid(True, which="both", ls="-", alpha=0.3)
36 | plt.legend()
37 | # ...
38 | # =====

```

15 Выводы

В результате выполнения лабораторной работы удалось познакомиться с законом Ципфа, а также тем, для чего он применяется. Сравнивая график для полученного корпуса и график закона Ципфа можно считать, что результаты, пусть и примерно, но соответствуют этому закону, особенно учитывая размер корпуса и узость предметной области игровой журналистики.

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №6 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №6 «Булев индекс»

Требуется реализовать процесс построения булева индекса для коллекции документов на основе ранее проведенной подготовки корпуса, токенизации, стемминга. Булев индекс должен представлять собой структуру данных, отображающую каждый уникальный токен на список идентификаторов документов, в которых этот токен встречается.

Необходимо разработать и описать следующие компоненты:

- Формат хранения инвертированного индекса (структура данных в памяти и на диске);
- Алгоритм построения индекса из подготовленного корпуса токенизированных документов;
- Способ сериализации (сохранения) индекса для долговременного хранения и десериализации (загрузки).

16 Описание

Булев индекс представляет собой инвертированную структуру данных, которая отображает уникальные термины на списки идентификаторов документов (номера документов, в которых встречается термин).

Основная задача индекса — обеспечить быстрый поиск документов по терминам с возможностью выполнения булевых операций. В данной работе реализовано построение индекса с использованием алгоритма **SPIMI (Single-Pass In-Memory Indexing)**, что позволяет эффективно обрабатывать коллекции документов большего объема, чем доступно оперативной памяти.

Формат хранения инвертированного индекса. Имеется два уровня представления:

1. **В памяти:** Для временного хранения терминов и их постлистов используется хэш-таблица фиксированного размера с цепочками для разрешения коллизий. Каждый элемент таблицы содержит термин (строка), односвязный список идентификаторов документов и указатель на следующий элемент.
2. **На диске:** Финальный индекс сохраняется в бинарном файле с определенной структурой, состоящей из заголовка, словаря терминов и секции постлистов.

17 Исходный код

Формат хранения инвертированного индекса. Имеются два уровня представления:

1. **В памяти:** Для временного хранения терминов и их постлистов используется хэш-таблица фиксированного размера с цепочками для разрешения коллизий. Каждый элемент таблицы содержит термин (строка), односвязный список идентификаторов документов и указатель на следующий элемент.
2. **На диске:** Финальный индекс сохраняется в бинарном файле с определённой структурой, состоящей из заголовка, словаря терминов и секции постлистов.

Алгоритм построения индекса SPIMI.

1. **Чтение коллекции документов:** Файлы с расширением .txt в указанной директории сортируются по числовому идентификатору, извлеченному из имени файла.
2. **Индексация:** Для каждого документа выполняется чтение токенов. Для каждого уникального в рамках документа токена добавляется запись в индекс: термин ассоциируется с идентификатором текущего документа. Подсчитывается потребление памяти структурой индекса.
3. **Запись промежуточного блока (индекса):** При достижении заданного лимита памяти текущее состояние индекса (хэш-таблица) сортируется по терминам в лексикографическом порядке и записывается на диск в виде отдельного блока (файл 'block_X.bin'). Формат блока: количество терминов, затем для каждого термина — длина строки, сам термин, количество документов в постлисте и список идентификаторов документов.
4. **Слияние блоков:** После обработки всего корпуса все промежуточные блоки сливаются в единый индекс. На каждом шаге выбирается лексикографически наименьший термин среди текущих терминов всех блоков, объединяются его постлисты из всех блоков (удаляются дубликаты, сохраняется сортировка по возрастанию ID из-за того что обрабатывали документы по очереди), и результат записывается в финальный индекс. Этот процесс повторяется до исчерпания всех терминов во всех блоках.
5. **Формирование финального файла:** Финальный индекс записывается в файл 'final_index.bin'. Промежуточные блоки удаляются.

```
kirill@LAPTOP-G4PP260P: /mnt/d/testing/src/bin
Обработка документа 35257 (../../../../prepared_texts/35257.txt)
Обработка документа 35258 (../../../../prepared_texts/35258.txt)
Обработка документа 35259 (../../../../prepared_texts/35259.txt)
Обработка документа 35260 (../../../../prepared_texts/35260.txt)
Обработка документа 35261 (../../../../prepared_texts/35261.txt)
Обработка документа 35262 (../../../../prepared_texts/35262.txt)
Обработка документа 35263 (../../../../prepared_texts/35263.txt)
Обработка документа 35264 (../../../../prepared_texts/35264.txt)
Обработка документа 35265 (../../../../prepared_texts/35265.txt)
Обработка документа 35266 (../../../../prepared_texts/35266.txt)
Обработка документа 35267 (../../../../prepared_texts/35267.txt)
Обработка документа 35268 (../../../../prepared_texts/35268.txt)
Обработка документа 35269 (../../../../prepared_texts/35269.txt)
Обработка документа 35270 (../../../../prepared_texts/35270.txt)
Обработка документа 35271 (../../../../prepared_texts/35271.txt)
Обработка документа 35272 (../../../../prepared_texts/35272.txt)
Обработка документа 35273 (../../../../prepared_texts/35273.txt)
Обработка документа 35274 (../../../../prepared_texts/35274.txt)
Сохранение последнего блока 11
Слияние 12 блоков...
Общее количество терминов в блоках: 756660
Финальный индекс сохранен в final_index.bin
Словарь содержит 251013 уникальных терминов
Построение индекса завершено
Время построения индекса) 850.4871 секунд
kirill@LAPTOP-G4PP260P:/mnt/d/testing/src/bin$
```

Структура бинарного файла финального индекса.

+-----+ <--Начало файла (offset 0)	
Заголовок (29 байт)	

Сигнатура (0x42584449)	# 4 байта,uint32_t
Версия формата (1)	# 1 байт,uint8_t
Размер словаря (N)	# 8 байт,uint64_t
Смещение на начало постлистов	# 8 байт,uint64_t
Размер секции постлистов в байтах	# 8 байт,uint64_t
+-----+	
Словарь терминов	

Термин 1:	
Длина термина (L1)	# 4 байта,uint32_t
Символы термина (L1 байт)	# сам термин
Смещение до постлиста (O1)	# 8 байт,uint64_t
Количество документов (C1)	# 4 байта,uint32_t

Термин 2:	
Длина термина (L2)	
Символы термина (L2 байт)	
Смещение до постлиста (O2)	
Количество документов (C2)	
...	
+-----+ <--Смещение postings_start	
Секция постлистов	

Постлист 1:	
ID документа 1.1	# 4 байта,uint32_t
ID документа 1.2	# 4 байта
... (всего C1 записей)	

Постлист 2:	
ID документа 2.1	# 4 байта
ID документа 2.2	# 4 байта
... (всего C2 записей)	
...	
+-----+ <--Конец файла	

Сериализация и десериализация индекса.

- **Сериализация (сохранение).** Финальный индекс формируется в процессе слияния блоков. Заголовок записывается в начале файла, за ним следует словарь, где каждый термин сопровождается метаданными: длина термина, сам термин, смещение относительно начала секции постлистов (не начала файла) и количество документов в постлисте. Постлисты записываются в отдельную секцию, на начало которой указывает поле 'postings_start' в заголовке.

```
1 void build_index( const char * input_dir )
2 {
3     printf( "Начало построенияиндекса...\n" );
4     printf( "Входная директория: %s\n", input_dir );
5
6     int file_count = 0;
7     char ** files = get_sorted_files( input_dir, &file_count );
8     if ( file_count == 0 )
9     {
10         printf( "В директориинет.txt файлов\n" );
11         return;
12     }
13
14     printf( "Найдено %d файлов\n", file_count );
15
16     SPIMIIndex * index = create_spimi_index();
17     int block_id = 0;
18     char ** block_files = NULL;
19     int block_count = 0;
20
21     for ( int i = 0; i < file_count; i++ )
22     {
23         uint32_t doc_id = extract_doc_id( files[i] );
24         printf( "Обработка документа%u (%s)\n", doc_id, files[i] );
25
26         FILE * file = fopen( files[i], "r" );
27         if ( !file )
28         {
29             fprintf( stderr, "Ошибка открытияфайла: %s (errno: %d)\n",
30                     files[i], errno );
31             continue;
32         }
33     }
```

```

32
33     char token[MAX_TERM_LEN];
34     char unique_tokens[1000][MAX_TERM_LEN];
35     int unique_count = 0;
36
37     while ( fscanf( file, "%255s", token ) == 1 )
38     {
39         int is_unique = 1;
40         for ( int j = 0; j < unique_count; j++ )
41         {
42             if ( strcmp( token, unique_tokens[j] ) == 0 )
43             {
44                 is_unique = 0;
45                 break;
46             }
47         }
48
49         if ( is_unique )
50         {
51             if ( unique_count < 1000 )
52             {
53                 strcpy( unique_tokens[unique_count], token );
54                 add_posting( index, token, doc_id );
55                 unique_count++;
56             }
57         }
58     }
59
60     fclose( file );
61
62     if ( index->mem_usage > MEMORY_LIMIT )
63     {
64         printf( "Достигнут лимит памяти, сохранение блока%d\n",
65                block_id );
66         write_spimi_block( index, block_id );
67
68         block_files = realloc( block_files, ( block_count + 1 ) *
69                                sizeof( char * ) );
70         block_files[block_count] = malloc( 50 );
71         sprintf( block_files[block_count], "block_%d.bin",
72                 block_id );

```

```

70         block_count++;
71
72         free_spimi_index( index );
73         index = create_spimi_index();
74
75         block_id++;
76     }
77 }
78
79 if ( index->count > 0 )
80 {
81     printf( "Сохранение последнего блока%d\n", block_id );
82     write_spimi_block( index, block_id );
83
84     block_files = realloc( block_files, ( block_count + 1 ) *
85         sizeof( char * ) );
86     block_files[block_count] = malloc( 50 );
87     sprintf( block_files[block_count], "block_%d.bin", block_id )
88     ;
89     block_count++;
90 }
91
92 free_spimi_index( index );
93
94 for ( int i = 0; i < file_count; i++ )
95 {
96     free( files[i] );
97 }
98 free( files );
99
100 if ( block_count > 0 )
101 {
102     printf( "Слияние %d блоков...\n", block_count );
103     merge_blocks( block_count );
104
105     remove_intermediate_files( block_count );
106
107     for ( int i = 0; i < block_count; i++ )
108     {
109         free( block_files[i] );
110     }
111 }

```



```

109     free( block_files );
110 }
111
112     printf( "Построение индекса завершено\n" );
113 }

```

- **Десериализация (загрузка).** При загрузке индексного файла сначала проверяется сигнатура и версия формата. Затем читается словарь. Для ускорения поиска термина при выполнении запросов словарь загружается не только в массив (для последовательного доступа), но и в хэш-таблицу. Хэш-таблица использует простое хэширование с цепочками, что обеспечивает амортизированное время поиска $O(1)$. Каждый элемент хэш-таблицы содержит термин, смещение до его постлиста и количество документов. Сами постлисты остаются на диске и считываются по запросу при помощи функции 'get_postings', которая перемещается к нужному смещению в файле и читает указанное количество ID документов.

```

1 IndexHandle * load_index( const char * file_path )
2 {
3     IndexHandle * handle = malloc( sizeof( IndexHandle ) );
4     handle->file_path = strdup( file_path );
5     handle->file_handle = NULL;
6     handle->dictionary_array = NULL;
7     handle->dict_size = 0;
8
9     FILE * file = fopen( file_path, "rb" );
10    if ( !file )
11    {
12        perror( "Ошибка открытия индекса" );
13        free( handle->file_path );
14        free( handle );
15        return NULL;
16    }
17
18    uint32_t magic;
19    uint8_t version;
20    uint64_t dict_size;
21
22    fread( &magic, sizeof( uint32_t ), 1, file );
23    fread( &version, sizeof( uint8_t ), 1, file );
24

```

```

25     if ( magic != MAGIC_NUMBER || version != 1 )
26     {
27         fprintf( stderr, "Неверный формат файла индекса: magic=%x,
                version=%d\n", magic, version );
28         fclose( file );
29         free( handle->file_path );
30         free( handle );
31         return NULL;
32     }
33
34     fread( &dict_size, sizeof( uint64_t ), 1, file );
35     fread( &handle->postings_start, sizeof( uint64_t ), 1, file );
36
37     uint64_t postings_size;
38     fread( &postings_size, sizeof( uint64_t ), 1, file );
39
40     if ( dict_size > 10000000 )
41     {
42         fprintf( stderr, "Слишком большое количество терминов: %lu. Файл поврежден?\n", dict_size );
43         fclose( file );
44         free( handle->file_path );
45         free( handle );
46         return NULL;
47     }
48
49     printf( "Загрузка словаря из %lu терминов...\n", dict_size );
50
51     uint32_t hash_table_size = ( uint32_t ) ( dict_size * 2 );
52     if ( hash_table_size < 1024 ) hash_table_size = 1024;
53     handle->hash_table = create_dict_hash_table( hash_table_size );
54
55     handle->dictionary_array = malloc( dict_size * sizeof(
        DictionaryEntry ) );
56     handle->dict_size = ( uint32_t ) dict_size;
57
58     for ( uint64_t i = 0; i < dict_size; i++ )
59     {
60         uint32_t term_len;
61         if ( fread( &term_len, sizeof( uint32_t ), 1, file ) != 1 )
62         {

```

```

63         fprintf( stderr, "Ошибка чтения длины термина%lu\n", i );
64         fclose( file );
65         free_dict_hash_table( handle->hash_table );
66         free( handle->dictionary_array );
67         free( handle->file_path );
68         free( handle );
69         return NULL;
70     }
71
72     if ( term_len > MAX_TERM_LEN )
73     {
74         fprintf( stderr, "Термин %lu слишком длинный: %u (макс: %d)
75             \n", i, term_len, MAX_TERM_LEN );
76         fclose( file );
77         free_dict_hash_table( handle->hash_table );
78         free( handle->dictionary_array );
79         free( handle->file_path );
80         free( handle );
81         return NULL;
82     }
83
84     char * term = malloc( term_len + 1 );
85     if ( fread( term, 1, term_len, file ) != term_len )
86     {
87         fprintf( stderr, "Ошибка чтения термина%lu\n", i );
88         free( term );
89         fclose( file );
90         free_dict_hash_table( handle->hash_table );
91         free( handle->dictionary_array );
92         free( handle->file_path );
93         free( handle );
94         return NULL;
95     }
96     term[term_len] = '\0';
97
98     uint64_t offset;
99     uint32_t count;
100
101     if ( fread( &offset, sizeof( uint64_t ), 1, file ) != 1 )
102     {
103         fprintf( stderr, "Ошибка чтения offset для термина%lu\n", i

```

```

103         );
104         free( term );
105         fclose( file );
106         free_dict_hash_table( handle->hash_table );
107         free( handle->dictionary_array );
108         free( handle->file_path );
109         free( handle );
110         return NULL;
111     }
112     if ( fread( &count, sizeof( uint32_t ), 1, file ) != 1 )
113     {
114         fprintf( stderr, "Ошибка чтения count для термина %lu\n", i )
115             ;
116         free( term );
117         fclose( file );
118         free_dict_hash_table( handle->hash_table );
119         free( handle->dictionary_array );
120         free( handle->file_path );
121         free( handle );
122         return NULL;
123     }
124     dict_hash_table_insert( handle->hash_table, term, offset,
125         count );
126     handle->dictionary_array[i].term = term;
127     handle->dictionary_array[i].offset = offset;
128     handle->dictionary_array[i].count = count;
129 }
130
131 fclose( file );
132 handle->file_handle = NULL;
133
134 printf( "Индекс загружен. Хэш-таблица содержит %u терминов\n",
135     handle->hash_table->count );
136 printf( "Коэффициент заполнения: %.2f\n",
137     ( float ) handle->hash_table->count / handle->hash_table->
138         size );
139
140 return handle;

```

140 || }
||

18 Выводы

В ходе выполнения данной лабораторной работы была подробно рассмотрена начальная реализация обратного индекса, а так же учтен вопрос того, что делать, если весь индекс не получается построить или использовать для поиска, храня его полностью в оперативной памяти. Идея построения промежуточных индексов (блоков) и дальнейшего сливания их в единый индекс на диске показалась мне очень занимательной.

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №7 по курсу «Информационный поиск»

Студент: К. К. Тузова
Преподаватель: А. А. Кухтичев
Группа: М8О-409Б
Дата:
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №7 «Булев поиск»

Требуется реализовать механизм выполнения поисковых запросов на основе ранее построенного булева индекса. Система должна обрабатывать булевы запросы, составленные из терминов (токенов) с использованием логических операторов.

Необходимо разработать и описать:

- Формат представления поискового запроса
- Алгоритмы выполнения базовых операций над постинговыми списками (списками id документов):
 - Пересечение (оператор AND)
 - Объединение (оператор OR)
 - Приоритеты (скобки)
- Стратегию оптимизации порядка выполнения операций в сложных запросах.

19 Описание

Булев поиск представляет собой метод поиска документов по запросам, составленным из терминов и логических операторов AND, OR с возможностью группировки с помощью скобок. В данной работе реализован механизм выполнения булевых запросов на основе ранее построенного инвертированного индекса. Система преобразует запрос в инфиксной нотации (привычной нам) в обратную польскую нотацию (RPN) и вычисляет результат с использованием стека и операций над отсортированными списками ID документов.

Формат представления поискового запроса. Формат – это привычная нам запись логических выражений с бинарными операторами AND и OR и круглыми скобками для задания приоритета. Примеры запросов:

- apple AND orange
- apple OR banana
- (apple OR banana) AND orange

20 Исходный код

Алгоритм выполнения булева запроса.

1. **Токенизация запроса:** Исходная строка запроса разбивается на токены: термины, операторы AND, OR, скобки. Процесс игнорирует пробельные символы и учитывает регистр только для терминов (операторы не чувствительны к регистру). Проводится нормализация токенов и стемминг (при помощи ранее разработанных нами библиотек `normalizer_lib.c` и `stemmer_lib.c`).
2. **Преобразование в обратную польскую нотацию.** Запрос преобразуется в RPN (этот формат удобнее использовать компьютеру при разборе логических выражений) с использованием алгоритма сортировочной станции (Shunting-yard). Алгоритм использует выходную очередь и стек операторов. Приоритет операторов: AND выше чем операторов OR. Скобки изменяют стандартный порядок обработки. Результатом является последовательность токенов в RPN, где операторы следуют за своими операндами.
3. **Вычисление RPN выражения.** Вычисление производится с помощью списков ID документов. Алгоритм последовательно обрабатывает токены RPN:
 - Если токен — термин, загружается соответствующий ему постлист из индекса (с использованием хэш-таблицы для быстрого поиска) и помещается в стек.
 - Если токен — бинарный оператор (AND или OR), из стека извлекаются два верхних списка, выполняется соответствующая операция, результат помещается обратно в стек.

После обработки всех токенов в стеке остается один список — результат выполнения всего запроса.

Алгоритмы выполнения базовых операций. Они основаны на методе слияния, аналогичном используемому в сортировке слиянием. Оба алгоритма требуют, чтобы входные списки были отсортированы по возрастанию ID документов (что верно для построенного индекса).

- **Пересечение (AND).** Алгоритм строит список документов, содержащих оба термина. Используются два указателя, изначально установленные на начала списков. На каждом шаге сравниваются текущие элементы. Если ID равны, элемент добавляется в результат, и оба указателя сдвигаются. Если ID в первом списке меньше, сдвигается указатель первого списка, иначе — второго. Сложность алгоритма — $O(n + m)$, где n и m — длины списков.

- **Объединение (OR).** Алгоритм строит список документов, содержащих хотя бы один из терминов. Используется метод слияния с двумя указателями. На каждом шаге в результат добавляется меньший из текущих элементов, и сдвигается соответствующий указатель. При равенстве ID элемент добавляется один раз, и сдвигаются оба указателя. Сложность алгоритма – $O(n + m)$.

```

1 uint32_t * intersect_lists( uint32_t * list1, uint32_t count1,
2                             uint32_t * list2, uint32_t count2,
3                             uint32_t * result_count )
4 {
5     if ( count1 == 0 || count2 == 0 )
6     {
7         *result_count = 0;
8         return NULL;
9     }
10
11     if ( count1 > count2 )
12     {
13         uint32_t * temp_list = list1;
14         uint32_t temp_count = count1;
15         list1 = list2;
16         count1 = count2;
17         list2 = temp_list;
18         count2 = temp_count;
19     }
20
21     uint32_t * result = malloc( count1 * sizeof( uint32_t ) );
22     uint32_t i = 0, j = 0, k = 0;
23
24     while ( i < count1 && j < count2 )
25     {
26         if ( list1[i] == list2[j] )
27         {
28             result[k++] = list1[i];
29             i++;
30             j++;
31         }
32         else if ( list1[i] < list2[j] )
33         {
34             i++;
35         }

```

```

36         else
37         {
38             j++;
39         }
40     }
41
42     *result_count = k;
43     return result;
44 }
45
46 uint32_t * union_lists( uint32_t * list1, uint32_t count1,
47                         uint32_t * list2, uint32_t count2,
48                         uint32_t * result_count )
49 {
50     uint32_t * result = malloc( ( count1 + count2 ) * sizeof( uint32_t ) )
51     ;
52     uint32_t i = 0, j = 0, k = 0;
53
54     while ( i < count1 && j < count2 )
55     {
56         if ( list1[i] == list2[j] )
57         {
58             result[k++] = list1[i];
59             i++;
60             j++;
61         }
62         else if ( list1[i] < list2[j] )
63         {
64             result[k++] = list1[i];
65             i++;
66         }
67         else
68         {
69             result[k++] = list2[j];
70             j++;
71         }
72     }
73
74     while ( i < count1 )
75     {
76         result[k++] = list1[i++];

```

```

76     }
77
78     while ( j < count2 )
79     {
80         result[k++] = list2[j++];
81     }
82
83     *result_count = k;
84     return result;
85 }

```

Стратегия оптимизации порядка выполнения операций. В рамках алгоритмов пересечения и объединения реализована эвристика для операции AND: при пересечении двух списков более короткий список всегда помещается первым аргументом, что может незначительно ускорить обработку за счет более раннего завершения при наличии очень короткого списка.

Поиск термина в загруженном индексе. Изначально термин ищется в индексе через хэш-таблицу. При загрузке индекса в память строится хэш-таблица, в которую помещаются все термины словаря вместе с метаданными (смещение постлиста в файле и количество документов). Коллизии разрешаются методом цепочек. Это обеспечивает амортизированное время поиска. Заметим, что бинарный поиск мог бы быть применен к отсортированному массиву терминов, загруженному в память, что дало бы гарантированное время поиска $O(\log N)$ и потребовало бы меньше памяти (считаем скорость выполнения запроса приоритетной).

Пример. Использование полученной утилиты:

```
printf( "Построение индекса: printf( "Поиск по индексу: printf( "Булев поиск:
```

1. `./indexer <in_dir>` — построение индекса из корпуса, находящегося в указанной директории
2. `./indexer -search <термин>` – вывод ID документов, содержащих указанный термин
3. `./indexer -bool <запрос>` – выполнение бинарного поиска по указанному запросу

Результаты выполнения запроса соответствуют статьям об игре Marvel Rivals, которая действительно считается клоном проекта Overwatch.

```
kirill@LAPTOP-G4PP260P:/mnt/d/testing/src$ ./indexer -bool "клон AND overwatch"
Загрузка словаря из 251013 терминов...
Индекс загружен. Хэш-таблица содержит 251013 терминов
Коэффициент заполнения: 0.50
Булев поиск: 'клон AND overwatch'
Найдено 35 документов:
  Документ 414
  Документ 488
  Документ 780
  Документ 970
  Документ 2345
  Документ 3141
  Документ 3168
  Документ 3221
  Документ 3540
  Документ 3583
  ... и еще 25 документов
```

Документ 414.

Marvel Rivals: Превью самого удачного клона Overwatch Супергерои не умирают Алексей Лихачев По вселенной Marvel что только не выходило: и файтинги (Marvel vs. Capcom), и пошаговая тактика (Marvel's Midnight Suns), и карточная игра (Marvel Snap), и многочисленные мобильные развлечения. Но никто не брался за геройский шутер, хотя с таким количеством персонажей он рано или поздно должен был появиться. Успех Overwatch вдохновил многих разработчиков — оказалось, героям в таких играх необязательно носить с собой пушки и стрелять. И Marvel Rivals чуть ли не целиком копирует формулу Overwatch — зачастую такая характеристика используется в негативном ключе, но в данном случае писать об игре что-то плохое не хочется. Что-то знакомое Заимствования встречаются на каждом шагу. В командах здесь по шесть игроков, как в первой Overwatch.

21 Выводы

После построения индекса оставалось только разобраться с обратной польской нотацией и понять, как выполняются операции пересечения и объединения постингов. Эксперименты с полученным бинарным поиском показали, что группируя входные токены можно находить релевантные документы, как это показано в примере. Однако без более продвинутых алгоритмов вывести действительно наиболее подходящие статьи не представляется возможным – на выходе мы получем просто список ID, удовлетворяющих запросу.